

Universidad Carlos III de Madrid
Data Science Project



**Hewlett Packard
Enterprise**

FINAL REPORT

“Taxonomy classification of AI incidents”

Mohamed Afif Chifaoui
Alberto Castellano Macías
Alejandro Leonardo García Navarro
Álvaro Maza Montesinos
Óscar Montoya Amores

2023/2024

We want to express our gratitude to

Claudia Garmendia

María Ridruejo

Saravanan Natarajan

Harold Molina-Bulla

For their continuous support and guidance throughout the whole duration of this project, and for their helpful advice to overcome the obstacles we encountered in the different phases of our work.

TABLE OF CONTENTS

Introduction

Background

Laws and Regulations

Evolution of AI Incident Databases

More on the project's database

Methodology

Taxonomy Research

Data Preprocessing

Unsupervised Learning

Data Labeling

Fine-Tuning and Models

Approaches

Change of Taxonomy

Approaches with the new taxonomy

Inference

Visualization

AI Incident Classification

Classification graphs

WordCloud per class

Alternatives

Conclusions

Next steps

References

Introduction

The fast growth of Artificial Intelligence (AI) in the last few years has brought a wide variety of applications and tools that contributed to transformative changes across various domains. However, the deployment of these AI systems in real-world scenarios has also given rise to incidents and challenges, often stemming from inadequate system designs or insufficient testing procedures.

Our project, carried out with the collaboration of **Hewlett Packard Enterprise (HPE)** and **Universidad Carlos III de Madrid**, aims to address the need for robust incident classification within the world of AI applications. The main focus is to develop a system capable of categorizing incidents based on different taxonomies using textual descriptions of the events.

The foundation of this effort lies in comprehending and leveraging the data obtained from the AI Incident Database. This repository encapsulates a diverse range of incidents, offering different scenarios where AI systems have exhibited unexpected behavior or faced challenges.

Some of the key steps in the project include:

- Background research.
- Data exploration.
- Taxonomy research.
- Tagging methods.
- NLP model implementation.
- Accuracy statement.
- Model-fine tuning.
- Visualization.

Through these steps that will be described more in detail in the following sections, the project seeks to contribute to the development of an incident classification system for AI vulnerabilities, addressing these kinds of issues associated with real-world implementations and ensuring the ethical and reliable use of AI technologies.

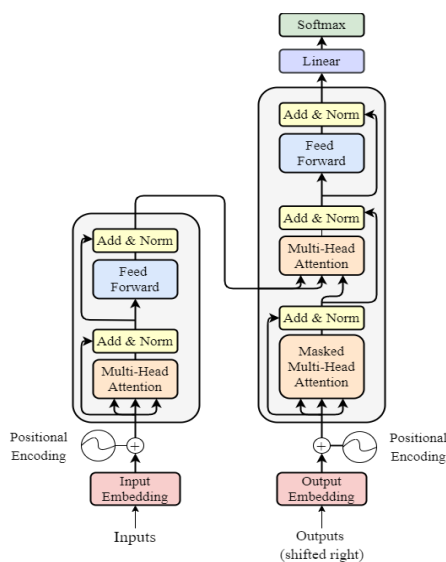
Language Models

Language Models (LMs) have undergone a transformative journey within the field of Natural Language Processing (NLP). Initially designed to predict the likelihood of a word given its context in a sentence, traditional LMs were limited in their scope and efficiency due to constrained data and computational capabilities.

The advent of Large Language Models (LLMs) represents a quantum leap in NLP capabilities. Characterized by their enormity, LLMs like GPT (Generative Pre-trained Transformer) or BERT (Bidirectional Encoder Representations from Transformers) possess millions to billions of parameters, enabling a deeper understanding of language nuances, context, and semantics.

Architecture and Mechanisms

LLMs are typically built upon Transformer architectures, which revolutionized NLP due to their attention mechanisms (**Figure 1**). These mechanisms allow the model to weigh the significance of each word/token concerning others in a sequence, facilitating the understanding of complex linguistic structures and context.



The self-attention mechanism in Transformers empowers LLMs to capture relationships between words regardless of their distance in a sequence, facilitating better comprehension of long-range dependencies and semantic intricacies.

During pre-training, LLMs are exposed to a massive corpora of text, absorbing patterns, syntax, and semantics inherent in language. Through unsupervised learning, these models learn to predict missing words, understand context, and generate coherent text by analyzing the relationships within the text corpus.

Figure 1: Transformer architecture

Applications in Incident Classification

In our project, the integration of LLMs, notably the "facebook/bart-large-mnli" and "facebook/bart-base," played a pivotal role in incident classification. Through fine-tuning, we adapted these models to understand and categorize incidents based on textual descriptions.

The process involved training these LLMs on our incident dataset, allowing them to discern patterns and contexts specific to incident descriptions. Leveraging their innate language understanding capabilities, we aimed to create a sophisticated classification system that accurately categorizes incidents into predefined taxonomies. The inherent ability of LLMs to capture nuances in language and contextual information contributes significantly to incident comprehension. By applying these models, our project sought to enhance incident management by offering a robust system capable of precisely categorizing incidents, aiding in their mitigation and analysis.

Background

Ethical Considerations and Potential Harms

The proliferation of AI applications across industries has brought forth ethical dilemmas. Discussions surrounding the responsible use of AI technologies have intensified, with concerns emerging over issues such as bias in algorithms, privacy infringements, and the implications of AI decision-making on societal norms and values.

The ethical implications extend to AI systems' potential to perpetuate or amplify existing social biases and prejudices, especially in areas like hiring, lending, and law enforcement. The opaque nature of some AI decision-making processes adds complexity to these concerns, as understanding and rectifying biases become crucial endeavors.

Moreover, the ethical debate encompasses the potential threats posed by AI systems, ranging from cybersecurity vulnerabilities to the existential risks associated with superintelligent AI. Discussions often revolve around ensuring AI systems prioritize safety, transparency, accountability, and align with human values to mitigate these risks.

Regulatory Frameworks and Governance

As AI technologies continue to permeate various facets of society, the need for regulatory frameworks and governance mechanisms becomes increasingly apparent. Policymakers and regulatory bodies are tasked with navigating the delicate balance between fostering innovation and safeguarding against potential harms.

Efforts are underway globally to establish guidelines, standards, and regulatory frameworks that govern the development, deployment, and usage of AI. These initiatives aim to address concerns related to data privacy, algorithmic transparency, accountability, and the societal impact of AI-driven decisions.

Balancing Benefits and Challenges

Amidst these debates and considerations, it's imperative to acknowledge the substantial benefits AI brings to society. AI advancements have fueled breakthroughs in healthcare, finance, transportation, education, and numerous other domains. Innovations such as predictive analytics in healthcare, autonomous vehicles, personalized learning, and efficient resource management showcase AI's potential to drive positive societal change.

The challenge lies in striking a balance between reaping the benefits of AI innovation and navigating its ethical and societal implications. Crafting policies and ethical frameworks that encourage innovation while mitigating risks becomes pivotal in fostering a responsible and beneficial AI landscape.

In essence, the current landscape of AI presents a compelling paradox, where the transformative potential coexists with profound ethical considerations and regulatory challenges. The ongoing discourse surrounding AI's ethical boundaries, societal impact, and regulatory frameworks underscores the need for conscientious and collaborative efforts to shape an AI-enabled future that prioritizes both innovation and societal well-being.

Laws and Regulations

The rapid advancement of artificial intelligence (AI) has prompted a global conversation about the need for regulations to govern its ethical and responsible use. As AI technologies keep appearing in several aspects of society, concerns have arisen regarding issues such as privacy, bias, accountability, and the potential impact on employment. Recognizing the transformative nature of AI, governments around the world have embarked on efforts to establish regulatory frameworks that balance innovation with safeguards against unintended consequences.

Artificial Intelligence Act

In Europe, the conversation around AI regulations gained momentum as policymakers sought to address the complex challenges posed by this technology. The European Union (EU) emerged as a frontrunner in shaping AI governance, emphasizing the importance of human-centric AI that respects fundamental rights and values. In April 2018, the European Commission took a significant step by releasing the "**Ethics Guidelines for Trustworthy AI**" [1], which laid out key principles, including transparency, accountability, and societal benefit. These guidelines marked the initial stages of regulatory thinking and set the stage for more comprehensive legislation.

The culmination of these efforts came to fruition with the release of the "**Proposal for a Regulation Laying Down Harmonized Rules on Artificial Intelligence**" [2] by the European Commission in April 2021. This ambitious legislative proposal aimed to create a unified framework for AI regulation across the EU member states. The proposed regulation outlined a risk-based approach, categorizing AI systems into high-risk and low-risk categories, each subject to specific requirements and obligations. High-risk AI applications, such as those in critical infrastructure, healthcare, and law enforcement, faced tougher rules to ensure safety and ethical considerations.

The proposed regulation also included provisions for transparency, human oversight, and data governance, underscoring the EU's commitment to fostering trustworthy and responsible AI development. The legislative initiative aimed to strike a balance between fostering innovation and safeguarding individuals and society from potential harm, reflecting the EU's dedication to becoming a global leader in shaping the future of AI regulation. As the proposal moved through the legislative process, it sparked discussions worldwide about the evolving landscape of AI governance and the significance of striking the right balance between innovation and ethical considerations.

The continued efforts made by the European Union leaders resulted in the groundbreaking **AI Act** [3], proposed by the European Commission in December 2023, which represents a milestone in global AI regulation, marking the first comprehensive law aimed at governing artificial intelligence. Recognizing the diverse risks associated with AI applications, the Act introduces a tiered approach, categorizing AI systems into different risk levels and outlining specific obligations for providers and users.

At the highest level of risk, AI systems deemed to pose an "**unacceptable risk**" are definitely banned due to their potential threat to individuals. These include systems engaged in cognitive behavioral manipulation, such as voice-activated toys encouraging dangerous behavior in children, social scoring based on personal characteristics, and real-time biometric identification systems, including facial recognition. Some exceptions are allowed for law

enforcement purposes, with limited use of real-time biometric identification systems permitted in serious cases and post-identification systems requiring court approval for prosecution of serious crimes.

Moving to the category of "**high risk**", AI systems that could negatively impact safety or fundamental rights fall into two subcategories. The first includes systems integrated into products covered by the EU's product safety legislation, spanning industries like toys, aviation, automobiles, medical devices, and lifts. The second subcategory encompasses AI systems in critical areas, such as the management of critical infrastructure, education, employment, law enforcement, and migration control. All high-risk AI systems, regardless of category, must undergo assessment before entering the market and be continuously evaluated throughout their lifecycle.

It also addresses the transparency requirements for different types of AI systems. Generative AI, including models like ChatGPT, must disclose that the content is AI-generated, design models to prevent the generation of illegal content, and publish summaries of copyrighted data used for training. High-impact general-purpose AI models, such as the advanced GPT-4, would face thorough evaluations, and any serious incidents must be reported to the European Commission.

For AI systems categorized as "**limited risk**", minimal transparency requirements are mandated, allowing users to make informed decisions. Users interacting with applications, especially those generating or manipulating image, audio, or video content like deep fakes, should be aware that they are interacting with AI. This approach seeks to strike a balance between transparency and usability in less risky AI applications.

This new and revolutionary law will not take effect until at least 2025, but it still positions the EU far ahead in the race with US, China and other powerful countries for being the sole leaders and pioneers in AI regulation.

2023 Executive Order on Artificial Intelligence for Congress

President Joe Biden's executive order, issued on October 30, 2023, outlines a comprehensive strategy for the safe, secure, and trustworthy development of Artificial Intelligence (AI) in the United States. The Executive Order (EO) signifies a coordinated effort across federal agencies to establish regulatory principles, standards, and guidelines, responding to the need for responsible AI development. It positions the U.S. to lead in global AI policy, particularly amid regulatory initiatives in the European Union and the United Kingdom, as well as increasing competition from China.

The EO underscores the importance of a collective effort involving government, the private sector, academia, and civil society to harness the benefits of AI while mitigating associated risks. This approach aligns with voluntary commitments made by leading technology companies earlier in the year, emphasizing a society-wide endeavor to address challenges such as fraud, discrimination, bias, and disinformation.

While the EO includes specific premarket testing and reporting requirements for certain AI developers, much of its regulatory impact is expected to arise from the generation of technical standards and guidance. Various executive departments and independent agencies are tasked with developing these standards over the next year. The standards, intended to influence the private sector through government purchasing power, cover a broad spectrum of sectors,

including financial services, healthcare, biotechnology, energy, transportation, telecommunications, intellectual property, competition, labor, education, housing, law enforcement, consumer protection, cybersecurity, national security, privacy, and trade.

The EO directs multiple regulatory authorities, including the Departments of Commerce, Energy, Health and Human Services, Homeland Security, Transportation, and Education, to address risks and potential benefits associated with AI in various sectors. Agencies such as the Federal Trade Commission (FTC), Consumer Financial Protection Bureau (CFPB), and Federal Communications Commission (FCC) are encouraged to take specific actions under the order. For instance, the Treasury Department is directed to issue a public report on best practices for managing AI-specific cybersecurity risks in financial institutions.

Recognizing the novel IP challenges posed by AI, the EO instructs the Patent and Trademark Office to publish guidance on inventorship and AI use. It also highlights the importance of innovation and competition in the AI market, encouraging the FTC to regulate unfair and deceptive practices. The order commits the U.S. to collaborate with other nations in advancing responsible AI development, reflecting a global perspective on AI governance.

Evolution of AI Incident Databases

The landscape of AI incident databases has evolved significantly, emphasizing the importance of documenting, understanding, and learning from AI-related mishaps. These databases have become instrumental in shedding light on real-world implications, fostering improvements, and guiding responsible AI development.

Recent advancements focus on detailed incident reports encompassing a broad spectrum of issues, including legal, ethical, and societal concerns stemming from AI deployment. This comprehensive documentation aids in accumulating a vast repository of AI failings, providing valuable insights into potential risks and pitfalls.

Open-source platforms for incident reporting have encouraged collaborative efforts, enabling contributions from diverse stakeholders. This inclusivity fosters a collective learning environment, allowing individuals, companies, researchers, and institutions to share incidents, thus enriching the database with varied perspectives and experiences.

However, the growth of AI incident databases comes with its set of challenges and ethical considerations:

1. **Quality Assurance:** The open nature of these databases allows for unrestricted submissions, potentially leading to inaccurate, redundant, or biased reports. Ensuring the accuracy and reliability of the data becomes a crucial challenge in training robust classification models and deriving meaningful insights.
2. **Transparency and Corporate Interests:** Dominance of incidents reported by major tech conglomerates raises questions about transparency and selective reporting. Smaller entities might refrain from sharing incidents due to potential reputational damage, creating a skewed representation within the database.
3. **Balancing Transparency and Privacy:** The debate surrounding the extent of transparency required from companies regarding their AI incidents continues. Striking a balance between encouraging transparency for collective learning and respecting corporate privacy remains an ongoing challenge.

More on the project's database

The AI Incident Database [4] is a specialized resource dedicated to indexing the collective history of harms or near harms caused in the real world by the deployment of artificial intelligence systems. This database aims to help us learn from past experiences to prevent or mitigate negative outcomes.

This database has some key features that may be worth mentioning:

1. **Incident Reports:** It includes detailed reports of various incidents involving AI. These reports cover a wide range of issues, from legal mishaps, such as a Colorado Springs attorney citing fake cases generated by AI in court documents, to ethical concerns, like South Korean AI chatbot pulled from Facebook after hate speech towards minorities.
2. **Submission and Indexing:** Users are invited to submit incident reports. These submissions are then indexed and made public. This often helps in accumulating a large record of AI failings, which may be crucial for learning and improvement.
3. **Entities and Taxonomies:** The database categorizes incidents and involved entities, providing insights into the most common types of incidents and the organizations associated with them. The different taxonomies proposed in the database web are 'harm' oriented, creating taxonomies unable to classify all the incidents and reports in any of them. Our work will be 'risk' oriented, taxonomy capable of including any incident in one of the classes.

- 4. Research and Learning:** By documenting AI incidents, the database serves as a valuable resource for researchers in the field of artificial intelligence. It helps in understanding the real-world impacts of AI and guides the development of more responsible AI systems.

Overall, it represents a significant step towards understanding and mitigating the risks associated with the deployment of these technologies in various sectors. By providing a centralized platform for documenting these kinds of incidents, it contributes to the development of safer and more ethical systems.

A very important aspect of this database is that it is open-source and anyone can submit their own AI incident reports and contribute to the increase of useful data so that everyone can have unlimited access to it. It is a way of learning and further preventing the occurrence of these types of incidents which may be very harmful in a large number of ways to individuals, societies, companies or institutions.

This presents a potential issue: since anyone can upload their AI incident reports, there's a risk of incorrect or redundant reports. This could make training and deploying an accurate classification model more challenging.

Also, another aspect to consider is that the most common entities that appear in the database are very big companies such as Facebook, Tesla and Google, so maybe these incidents are being shared with a hidden interest of these companies, and not just for full legitimate purposes. In other words, these companies might be sharing only some incidents that they have experienced and not other incidents that they would want to keep secret as it might harm their public image.

In addition, some small companies are probably not attracted to the idea of sharing their AI incidents as this might generate a damaging image of the company and they might be scared of losing potential customers due to this.

There's an ongoing debate about whether companies should be transparent and share extensive details about their AI incidents for the greater good. Our project aims to support this by promoting the dissemination of information. This will help in preventing and mitigating such incidents more effectively.

Methodology

As stated in the key steps described in the [introduction](#), there were certain steps/guidelines that were given to develop this project. Clearly, one of the first things that had to be done was understanding the data. It comes from the “AI Incident Database” web page [4], which focuses on cataloging accidents caused by AI technologies in the real world using taxonomies.

Considering that this database is uploaded weekly, one may think that a great part must be well classified. However, the current situation is the one exposed in **Figure 2**. The web page allows users to have a spatial view of incident classifications, with different colors (each representing a taxonomy). Nonetheless, there is a vast amount of white ones, which correspond to unclassified incidents.

Due to this exact purpose, the main goal of this project is to try to build a system that sorts incidents into taxonomies using text about what happened (see the full code of the project [here](#))

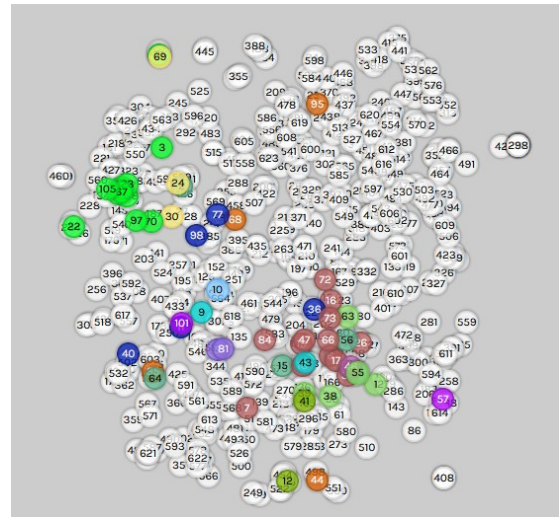


Figure 2: Space visualization of incident classifications from taxonomies

To achieve our goal, we focused on using the incident descriptions as our primary data source from the database. Since the original data lacks labels, there is no need for additional information. Furthermore, the remaining columns only contain details about the incident authors, dates, companies associated with the incidents, and a title, all of which we chose not to utilize.

Initially, our plan was to utilize the report descriptions, which amounted to 3028 rows, in order to have a larger dataset. However, we observed that the incidents dataset organized the reports into incident descriptions, grouping similar or identical reports under the same description. Consequently, we ultimately opted to work with the incidents dataset, despite its smaller size (556 rows). We believed it was more valuable to have a concise dataset with well-defined and clear descriptions.

Taxonomy Research

Once the data was fully understood, it was time to start the taxonomy research. This involves studying and creating structured classifications to organize complex information into categories that share similar characteristics. This process is essential when you need to develop taxonomies, as it allows you to draw inspiration and insights from existing classification systems. In particular, three taxonomies were deeply investigated:

1. **Taxonomy of AI Risk:** This taxonomy was developed by the NIST (“National Institute of Standards and Technology”) and considers (a) *technical design attributes* (factors that are under the direct control of system designers and developer), (b) *how AI systems are perceived* (any assessment of an AI system that is made by a human falls into this category) and (c) *guiding policies and principles* (rules about things in our society that we care about) [5].
2. **AI Harm Taxonomy:** Developed by the CSET (“Center for Security and Emerging Technology”) and distinguishes between *tangible* (observable) and *intangible* harm (harm that cannot, even with additional information, be observed) [6].
3. **Sociotechnical Harms of Algorithmic System:** Presented by several researchers that propose a taxonomy of sociotechnical harms. It contains classes along the lines of: representational harms, allocative harms, Quality-of-Service harms, interpersonal harms and social system harms [7].

Once investigation was done, we were able to reach a consensus regarding what the best taxonomy would look like. The first proposed taxonomy (**Figure 3**) contains:

1. **Security and safety:** Covers issues related to the risks and dangers of technology, including autonomous vehicles' malfunctions, accidents involving self-driving cars and physical dangers such as theft or hazardous situations.
2. **Social harms and ethical violations:** Contains topics related to discriminatory, offensive, and harmful content that includes biases, hate speech, derogatory language, and unethical practices such as deepfakes and disinformation.
3. **Privacy violations:** Relates to breaches of personal data and confidentiality, including unauthorized surveillance, misuse of biometric data, identity theft, and any actions taken without consent that comprise an individual's privacy and rights.
4. **Operational incidents:** Issues with algorithmic systems and automated processes, including inaccuracies, biases, system vulnerabilities, cybersecurity threats such as malwares and mistakes in decision-making or recommendations, which can lead to operational failures or inefficiencies.

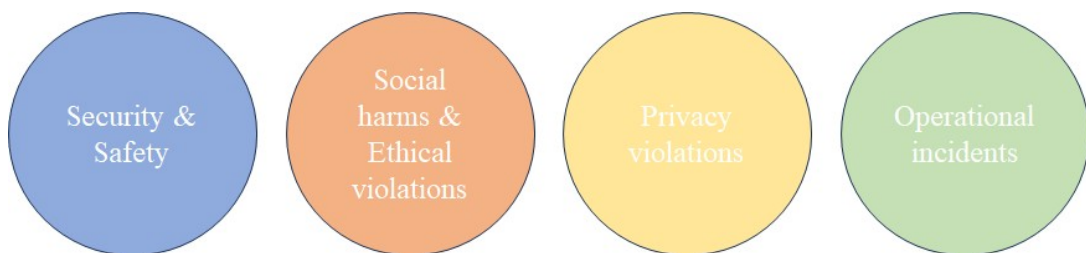


Figure 3: Proposed taxonomy review

Data Preprocessing

After pre-defining our taxonomies, we moved on to analyze in depth the content of the database, in order to decide what data is relevant for us and what data can be discarded. The database contains several datasets, mostly with information that is useless to our project.

Figure 4: Word Cloud containing the most frequent words in the dataset

3. **Text vectorization:** we vectorized all documents in our newly created corpus in order to perform helpful visualization techniques that will give us further insights of our corpus.
 - a. **Dictionary creation:** Created a dictionary from the words in our corpus using the gensim library [9].
 - b. **Filtering tokens:** Removed tokens that have either very few or too many appearances in our corpus, i.e. appear in less than 3 documents or in more than 95% of documents, as they are probably not very informative or could even be typos.
 - c. **Bag-Of-Words representation:** Obtained a numeral representation of our corpus and transformed any list of tokens into a list of tuples, which can be seen as a vector with the form (id, number of occurrences).
 - d. **TF-IDF model:** Representation of each word that measures how frequently each word appears in a document and how rare it is for a word to appear, giving higher scores for words that appear in fewer documents. They are represented as a tuple in the form (number of occurrences, rarity score).
4. **Topic modeling with LSI :** Discover the underlying themes that exist in the collection of our corpus by identifying patterns of words that appear together frequently (**Figure 5**). This helped us visualize some of the meaningful topics in our data before proceeding with the next step.

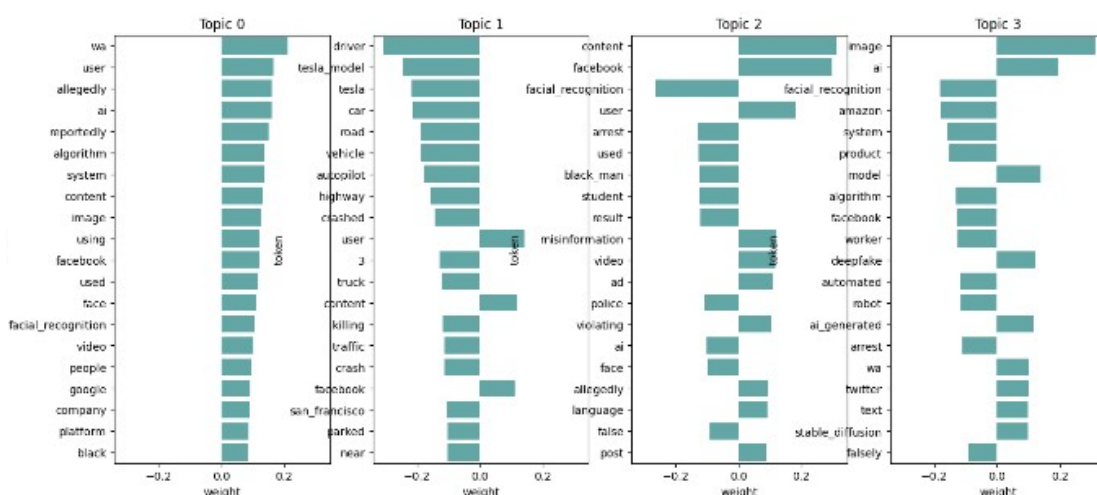


Figure 5: Topic modeling with LSI

During the natural language processing (NLP), we considered using some other tools such as topic modeling using other techniques such as LDA or text vectorization using Word2Vec, but for our purpose we considered that the already proposed techniques and tools were sufficient to achieve our goal of visualizing the patterns and categories that the text in our dataset represent.

As we can see from *Figure 4* and *Figure 5*, the purpose of this whole preprocessing process is to gain a general understanding of the most predominant words and patterns that appear in our dataset, which helped us in deciding the taxonomies we were going to use to train our model. This is relevant for us because we had to change our taxonomies several times as our model was not identifying well some instances and at times got confused.

Therefore, having a deeper understanding of our dataset and the types of taxonomies to which every reported AI incident belongs to is crucial to developing a successful model that is able to classify a large number of incidents correctly.

In fact, we were forced several times to adjust our proposed taxonomies as they were not as relevant and specific as we had anticipated, so these preprocessing steps played an important role in helping us reach a potential solution for choosing the taxonomies.

Now that we have performed a deep analysis of the words and text in our dataset, we proceeded with the creation and deployment of our model, after making some more adjustments in our vectorization techniques.

Unsupervised Learning

In addition to this exploration of the corpus to identify patterns and groups of descriptions or words, we also implemented **unsupervised learning** techniques, taking advantage of the work already done on the texts. The aim of this was checking if using unsupervised approaches, it was possible to visualize and divide incidents in groups, and use this information to define the labels and divide.

To work with the unsupervised learning functions we adapted the TF-IDF to a dense array, which is suitable for the methods to use. As our purpose was to detect groups, we made

use of different clustering algorithms such as *K-Means*, *Gaussian Mixture*, *AffinityPropagation*, *AgglomerativeClustering* and *OPTICS*. The point of trying these different techniques was in determining the way the descriptions could be organized, as each of them have different approaches to generate and detect groups of data.

Actually, we did not only try with the different techniques but also different number of clusters, as when this particular part of the project was being done, the taxonomies were not exactly defined yet, and one of the objectives was to determine the best number of labels to use in the taxonomy.

Finally, in order to visualize the results of the methods, we applied Principal Components Analysis (PCA), to perform a dimensionality reduction. On it, different kernels were used (*linear*, *rbf*, *sigmoid*, *polynomial*) to check if any of them could change the way of seeing the data points that represent each incident. Some of the resulting plots are given below:

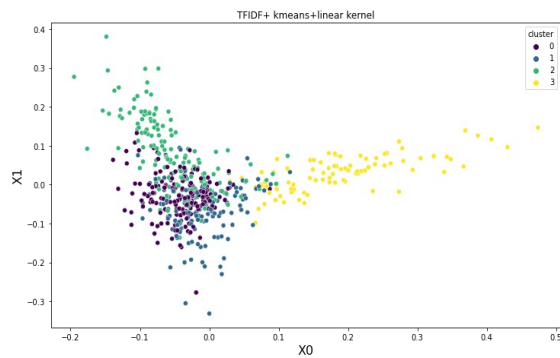


Figure 6: Linear kernel

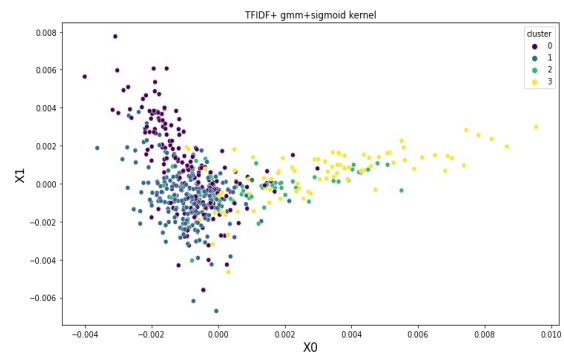


Figure 7: Sigmoid kernel

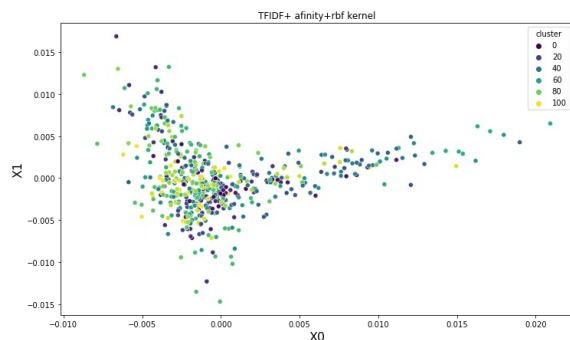


Figure 8: RBF kernel

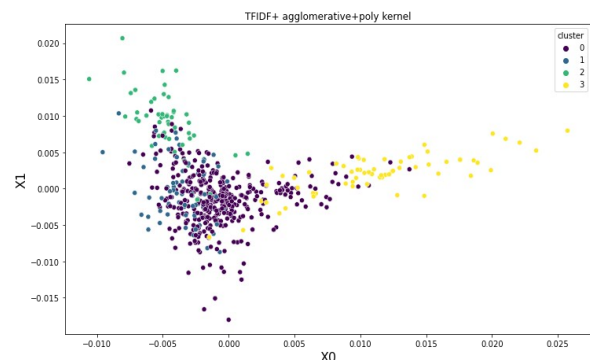


Figure 9: Polynomial kernel

Data Labeling

Once having cleared the taxonomies, the next crucial step was the **labeling of the dataset**, which consisted of assigning the correct label to its corresponding incident. As we have four taxonomies, we had to distribute 4 labels (*security and safety, social harms and ethical violations, privacy violation and operational*) into the whole dataset.

Together with the definition of the taxonomies, this step is one of the most crucial. Labeling the incidents properly is going to influence later the classification task and the scalability of the model in the future, so tagging constantly with the same approach and with the same metrics to decide for one label or another is a key point, as the classification will take this labels and trust that the data is labeled correctly for training.

To do this, a manual labeling approach was used, consisting of recognizing the most important words of each taxonomy group, storing them in a dictionary and linking the text in the descriptions to the label based on these important words that differentiate each taxonomy.

Regarding the classifier model, we will be using the **Zero-Shot Classification** from the AI company **Hugging Face** [10], which is a task in natural language processing where a model is trained on a set of labeled examples, but is then able to classify new examples from untrained data and without labels.

In this classifier, we provide the model with a sequence of text (in our case the description of incidents) and we can choose or not to give the model some labels. If we do not give labels, it will return the set of labels (in this case words) that it considers to be more related to the text, so it returns possible labels for a text which is unlabeled. In case we give the model some labels, it will return how much relation it seems between each label and the text.

In our case, we specifically chose two pre-trained models: **facebook/bart-large-mnli** [11] and **facebook/bart-base** [12]. First we started only using the “facebook/bart-large-mnli” model, but during the training and development of the model, we found it interesting to try a similar but smaller model. This is due to the exact purpose that our amount of data is not enormous, and the comparisons between the performance of fine-tuning both models could be useful.

The first approach to this method, zero-shot classification using facebook/bart-large-mnli, was just discovering how it works. We introduced some descriptions of our data and saw how, without giving the model any label as the data was not labeled yet, the classification returned the 3 words it considers more related to each description.

Having explained and understood this transformer, we used it for the first time in our data, as we wanted to check how well the labels have been assigned to each description with the dictionary. For this, we passed each of the descriptions to the zero-shot model, with our proposed taxonomy as labels, so it returns how much it links each of them to each description.

With this approach, we created a way of comparing the labels already assigned with the scores returned by the classification and observed if the one with highest score is the one selected by us with our manual approach using a dictionary of words.

Finally, a threshold was used on the scores of the zero-shot classification and manual checks of the labels. With this, it was desired to check and make sure that the labeling was correct in certain descriptions (those in which a low value is returned or those in which the label

with highest score is not the same that was selected by us). With this being done, and being sure that the labeling is correct, we can pass to the classification task.

It is known that the zero-shot classifier is built on top of a pre-trained language model, so the following step will be to fine-tune it by training it on our *incidents* dataset and dividing the task in train and test so that afterwards, we can proceed to evaluate its performance and obtain the correct classification rate.

Fine-Tuning and Models

To use as a guide, we implemented some basic common classification approaches previous to the fine-tune of the transformers. To sum up, we vectorize the incidents in TF-IDF format and pass them alongside their labels through a pipeline of classification models using GridSearch. The tried methods include: Random Forest, Gradient Boosting, Support Vector Classification, Naive Bayes, Logistic Regression, XGBoost and LightGBM.

For all of the methods we included a classification report to study accuracy, F1, recall, support and confusion matrix, for being able to compare the performances quickly. Even Though, as we have said, our objective was just having a guide of the performance of basic common methods, the results were actually quite surprising, with results from 0.60 to 0.69 accuracy, but with some labels clearly being more relevant than others.

Before being able to fine-tune a Hugging Face model, it is necessary not only to check that the model to fit is suitable for our objective, but also to adapt our data to the format needed by the transformer, which is not always the same.

In our specific situation, we encountered a requirement to modify how we assign labels to our data. The model we are using does not accept categorical labels, which are labels that are not in numerical form, such as words or categories. Instead, it requires numerical labels. To meet this requirement, categorical labels had to be converted into numerical ones.

Even though we had previously inspected and assigned labels to each description in our dataset, we decided to perform an additional check before proceeding with the label transformation, ensuring that there were no missing values in our dataset. This step is important because missing data could lead to errors during the label transformation process.

Once we confirmed that there were no missing values, we then proceeded with the transformation of the labels from their categorical form to numerical form, which is compatible with the model's requirements:

```
labels = df['label'].unique()
true_labels = df["label"] #save original labeling
id2label = {idx:label for idx, label in enumerate(labels)}
label2id = {label:idx for idx, label in enumerate(labels)}
df['label'] = df['label'].replace(label2id)
true_labels_num = df["label"]
```

As observed in the snippet of the code, the original labeling is saved before applying changes. An important step involves creating the *id2label* and *label2id* dictionaries, where the

keys represent the original label, and their corresponding numerical representations are the values. This allows us to simply replace each label with its corresponding number.

The reason why *id2label* and *label2id* are important to save is not only to make the process of recovering the original labels faster, but also because we have the option to save them alongside the rest of the fine-tuned final model for later use in any other document or notebook.

Transforming the labels into numbers is not the sole modification made to the data. We will elaborate on the data preprocessing when discussing the various approaches we have attempted to create a model. This way, each model will be comprehensively explained before moving on to the next one.

Approaches

Returning to the first tried approach, after having the 4 labels in numbers, a tokenization is needed. As happens in any other common NLP work, texts need to be transformed so models and computers can work with them. In this case, using transformers and fine-tune has the advantage of having a pre-defined tokenizer, so the transformation of the data has the same format of the one in the original model. The tokenizers are imported the same way as models:

```
tokenizer = AutoTokenizer.from_pretrained("facebook/bart-large-mnli")
```

After that, mapping each incident description to the tokenizer is enough to transform the data:

```
def tokenize_function(examples):  
    return tokenizer(examples["description"], padding="max_length",  
truncation=True)  
  
tokenized_df = df.map(tokenize_function, batched=True)
```

With this done, we can now pass without any problem to the fine-tuning task. As in any other approach, division in train and test, and even validation is done to the dataset. A key aspect to remember is that our models and the ones to fine-tune, do not necessarily have the same amount of labels, so when opening the model:

```
model =  
AutoModelForSequenceClassification.from_pretrained("facebook/bart-  
base", num_labels = 4, ignore_mismatched_sizes = True, id2label=id2label)
```

The **num_labels** parameter is crucial and should be set to the number of unique labels we have, which, in this case, is 4. This ensures that the model is properly configured to handle the specific number of labels in our dataset.

Next, before training the model, different values, parameters and even metrics can be changed and set to find the best possible performance. As we use the transformers library, we work with a **Trainer**, which has **TrainingArguments**:

```

training_args = TrainingArguments(
    "./our_model",
    evaluation_strategy="epoch",
    save_strategy="epoch",
    logging_steps=30,
    per_device_train_batch_size=2,
    save_total_limit=1,
    per_device_eval_batch_size=1,
    eval_accumulation_steps=1,
    learning_rate=1e-5, # Experiment with different values
    num_train_epochs=5,
    weight_decay=0.1, #to avoid overfitting
    load_best_model_at_end=True)

metric = load_metric("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred.predictions, eval_pred.label_ids
    # Ensure logits is a 2D array
    logits = logits[0] if isinstance(logits, tuple) else logits
    if logits.ndim > 1:
        predictions = np.argmax(logits, axis=-1)
    else:
        # This handles the case when logits is a 1D array
        predictions = np.round(logits).astype(int)
    # Calculate accuracy
    accuracy = accuracy_score(labels, predictions)
    # Calculate precision, recall, and F1-score for each class
    precision, recall, f1, _ = precision_recall_fscore_support(labels,
predictions, average='weighted')
    return {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1': f1
    }

trainer = Trainer(
    args=training_args,
    model=model,
    train_dataset=train_df,
    eval_dataset=eval_df,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,)

```

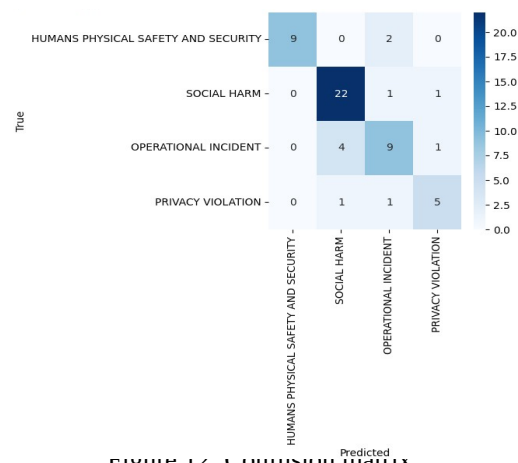
In this example, some of the parameters can be noticed. For example, we used epoch and a fixed learning rate (we changed it manually when checking the results after each epoch) and a weight decay that varies from 0.01 to 0.2 to avoid overfitting, which we suffered sometimes during the training of the models.

This first try was not great as it was the first time trying the transformer. Because of this, we spent some time working on which parameters to add, how to variate them, and understanding the results.

At first, we faced a challenge because our accuracy was only around 64%, which was not good enough for us. Our main goal has always been to achieve 90% accuracy. To assess how well our model was doing during training and testing, we looked at not just accuracy but also at other measures like precision, recall, and F1-Score for each step. F1-Score will be especially important in our future explanations of the next models.

At the end, after properly adapting the **learning rate** and **size** of the train and test datasets, 80% of accuracy was reached with this first approach. From our point of view, even though it took some time, it was a great guideline for future models.

Here you can observe a confusion matrix (**Figure 12**) extracted during the training of the model, showing the results of the classification in a test set. Not a bad classification at all, but the imbalanced data was probably starting to limit our model from our point of view.



Following, as commented above, we tried “facebook/bart-base” instead of “facebook/bart-large-mnli”. A more basic model, but this is not necessarily worse, since less amount of parameters to train could be a good option.

During our first try, as already explained, we suffered from drops caused by overfitting. Our data set is not so big, it is actually limited in size for now, so using a smaller model could reduce the overfitting problem.

Additionally, the training process was obviously faster, so all the process of finding the parameters that best suited both the fine-tuning of the model and the data was faster. Only by changing the pre-trained model, we improved the performance a little, going up to 80%, but with much less problems to train and avoiding overfitting more easily.

On the other hand, these metrics were not realistic at all, in the last steps of the training overfitting was still visible, and during different testings of the resulting model we could see it was not working well at all with the testing data and hand-written descriptions created by us. Our final conclusion was the same as with the first model, the unbalance was giving too much importance to the Social Harms label, and in addition, the truth is that some descriptions are really difficult to classify in just one label, as they can share characteristics of more than one.

The gained experience working with transformers and the obtained results made us search for a different approach. Actually, we considered 2 changes for re-doing the model and

labeling: Multi-Labeling Classification and a recomposition of the labels in 3 categories instead of 4.

First, before changing the taxonomy we opted to give a try to multi-label classification with the original 4 labels. Obviously, the training procedure and the data preparation suffered some changes.

Until now, the first models were based on multi-class classification, that is, one column to decide the label, in our case 4 labels. Multi-Label will change that to one column per label, having to classify each description for each of them, this is, predict if each column is True or False for that description, opening the door to descriptions with more than one label.

It is true that our objective is to classify just with one label each incident, but in this way the model will learn a binary classification for each label, this will help in those incidents with more than one possible label, as the model won't have to decide between labels having similar probabilities, just if the label corresponds or not to the incident independently for each of them.

Transforming the data is straightforward, as observed in **Figure 13**: we create a column for each label and mark it as 'True' if the incident corresponds to that label.

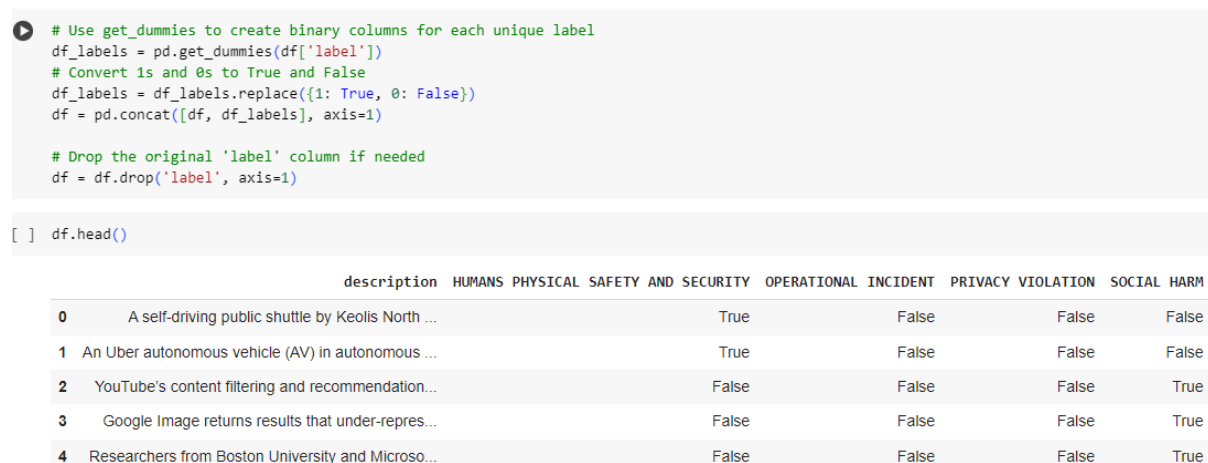


Figure 13: Data transformation

The difficulties come from adapting the pre-trained models to this task. Now, we do not tokenize the descriptions and pass the labels to numeric and that is all, we need to tokenize the data in a different way.

Based on tutorials that Hugging Face provides, like this one: [tutorial multi label text classification](#) [13], we tackled the problem of adapting the data. Now, while encoding the data with the tokenizer, we also encode the labels with it:

```
#Tokenize the data
def tokenize_data(examples):
    # take the text(descriptions) of the data
    text = examples["description"]
    # encode them using the model tokenizer exported
    encoding = tokenizer(text, padding="max_length", truncation=True,
```

```

max_length=248)
    # add labels, we take the label as key, and take its value for each
description and label
    labels_batch = {k: examples[k] for k in examples.keys() if k in labels}
    # create numpy array of shape (batch_size, num_labels) to store values
of the labels for all the batch
    labels_matrix = np.zeros((len(text), len(labels)))
    # fill numpy array
    for idx, label in enumerate(labels):
        labels_matrix[:, idx] = labels_batch[label]

    encoding["labels"] = labels_matrix.tolist()

    return encoding

tokenized_df = df.map(tokenize_data, batched=True,
remove_columns=df.column_names) #no need to keep the original columns

```

This now returns a dataset with *input_ids*, *attention_mask* and *labels*, (dict_keys(['input_ids', 'attention_mask', 'labels'])), being the labels a list of 1 or 0 depending on if the labels is True or False for each description. This is the reason why we still need the '*id2label*' list as in previous approaches, as now the id is the index position of each label in the list.

In addition, when opening the pre-trained model, now we need to specify we are working with a multi-labeling classification problem, so the model weights and sizes can be adapted from the pre-trained model to our objective:

```

model=AutoModelForSequenceClassification.from_pretrained("facebook/bart-
base", num_labels = 4, ignore_mismatched_sizes = True,
problem_type="multi_label_classification", id2label=id2label, label2id=labe
l2id)

```

Finally, we need to create a self-made metrics calculator, which we learnt from the same tutorial that we used for obtaining the information about how to tokenize for multi-labeling.

Why? Now, each label probability is calculated independently, so it is not as simple as before, where the model just took the label with higher probability as the predicted one.

The multi-labeling classification is intended to be able to return one or more true labels from the set of possible labels, even if our objective is to predict just one, the correct implementation of this method is to give room for the model to predict more than one label if it is the most likely option.

This is why even if each label probability is independent, a threshold is set in the set of probabilities, so the model trains to predict each label if it is higher or equal to that threshold, so now we have one more parameter to take care about, as putting a higher threshold will be more

restrictive but could result in no output but putting it too low can make the model not training properly.

To enter our own created metrics computation into the model, is enough to add the parameter 'compute_metrics' in the trainer with our function:

```
training_args = TrainingArguments(  
    "./our_model",  
    evaluation_strategy="epoch",  
    save_strategy="epoch",  
    logging_steps=30,  
    per_device_train_batch_size=2,  
    save_total_limit=1,  
    per_device_eval_batch_size=1,  
    eval_accumulation_steps=1,  
    learning_rate=1e-5, # Experiment with different values  
    num_train_epochs=5,  
    weight_decay=0.1, #to avoid overfitting  
    load_best_model_at_end=True,  
    metric_for_best_model="f1" #f1 is the metric to focus  
)  
  
trainer = Trainer(  
  
args=training_args,model=model,train_dataset=train_df,eval_dataset=eval_d  
f,  
    compute_metrics=compute_metrics,  
)
```

Afterwards, the *compute_metrics* function was adapted:

```
# source: https://jesusleal.io/2021/04/21/Longformer-multilabel-classification/  
def multi_label_metrics(predictions, labels, threshold=0.6):  
    # first, apply sigmoid on predictions which are of shape (batch_size,  
num_labels)  
    sigmoid = torch.nn.Sigmoid()  
    probs = sigmoid(torch.Tensor(predictions))  
    # next, use threshold to turn them into integer predictions  
    y_pred = np.zeros(probs.shape)  
    y_pred[np.where(probs >= threshold)] = 1 #if prob of a  
label>threshold, we predict the label and put a 1  
    # finally, compute metrics  
    y_true = labels  
    f1_micro_average = f1_score(y_true=y_true, y_pred=y_pred,  
average='micro')
```

```

roc_auc = roc_auc_score(y_true, y_pred, average = 'micro')
accuracy = accuracy_score(y_true, y_pred)
# return as dictionary
metrics = {'f1': f1_micro_average,
           'roc_auc': roc_auc,
           'accuracy': accuracy}
return metrics

def compute_metrics(p: EvalPrediction):
    preds = p.predictions[0] if isinstance(p.predictions,
        tuple) else p.predictions
    result = multi_label_metrics(
        predictions=preds,
        labels=p.label_ids)
    return result

```

It is really important to shift our focus towards the F1-Score, as it effectively integrates precision and recall. The reasoning behind this decision is clear.

Presently, each description has the potential to correspond to more than one label, whereas our data has been annotated with just one label per incident. If we were to rely solely on accuracy as our evaluation metric, any incident predicted with more than one label would be deemed incorrect. However, in such cases, the outcome could include the correct label along with an additional, albeit incorrect, one.

This is precisely why the F1-Score is the preferred choice. It incorporates both false negatives and false positives rates. In situations where there are multiple labels, the F1-Score ensures that the correct label is regarded as accurately classified, even if the model predicts another label incorrectly. This approach ensures a balanced assessment, penalizing the model for any incorrect predictions.

With respect to the results, we obtained a higher performance improving to 0.836 in the F1 score, which is good, but when checking the performance per label, the problem of the imbalance of performance between labels persisted:

Classification Report:				
	precision	recall	f1-score	support
0	0.93	0.81	0.87	16
1	0.86	0.94	0.90	34
2	0.80	0.76	0.78	21
3	0.80	0.62	0.70	13

It can be easily observed that the minority class corresponding to label 3, “**Privacy Violation**”, exhibits significantly lower statistics. This label, in particular, should be distinguishable quite effortlessly from the others, as the incidents pertaining to it are less reliant on interpretation and lend themselves to easier labeling. Consequently, the decision was made

to explore a new labeling approach, incorporating some adjustments, in order to achieve a more balanced dataset.

Change of Taxonomy

As we continued to analyze and encounter AI incidents, we recognized certain inherent similarities in the initial taxonomy and faced challenges related to class imbalances. In response to these observations and practical considerations, we streamlined our classification system to three broad categories (**Figure 13**):

1. **Human Physical Safety:** This category encompasses incidents related to the physical well-being of individuals, which could result from AI systems' actions or failures.
2. **Social Harms:** The second category consolidates aspects of the previous "Social Harms and Ethical Violations". It contains topics related to discriminatory, offensive, and harmful content that includes biases, hate speech, derogatory language, and unethical practices such as deepfakes and disinformation.
3. **Privacy, System Security, Misuse or Bugs:** This last label contains incidents related to data privacy, system security breaches, misuse, and technical glitches. By grouping these issues under a single umbrella, we aim to address the overlaps and commonalities that exist within them.



Figure 13: New taxonomy

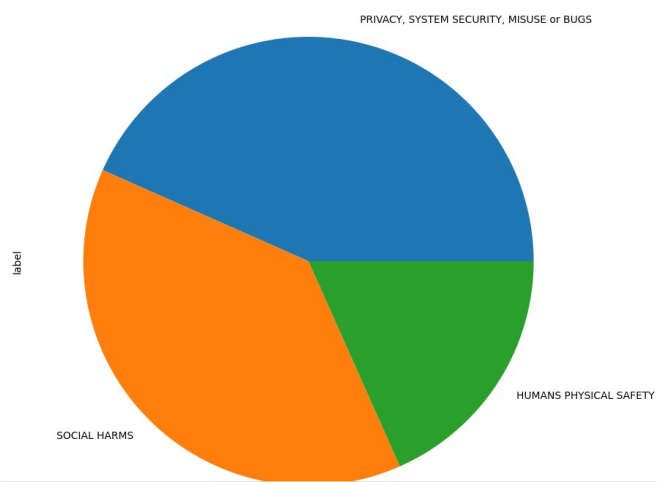
already

As we had the original labeling, we looked for the fastest way in which we can re-adapt the labels for the data frame, and we decided to use the python library "**label-studio**" [14]. It runs a software created for the labeling of data used for different tasks, and lets us create the new labels and select them manually just by clicking on the different new labels, having also as reference the older ones, which made the process easier and faster than expected.

```
PRIVACY, SYSTEM SECURITY, MISUSE or BUGS 241
SOCIAL HARMS 213
HUMANS PHYSICAL SAFETY 102
Name: label, dtype: int64
<axes: ylabel='label'>
```

The final distribution of the new labels is visible in **Figure 14**. It's easy to appreciate significant differences in terms of balance.

There was some concern about "Humans Physical Safety" having fewer



instances, but confidence remained that the model would effectively differentiate instances corresponding to this label, especially since they directly affect human physical safety.

Figure 14: Label distribution

Approaches with the new taxonomy

After applying the new taxonomy to all incidents, we were optimistic about improved results. However, we had concerns because the label "**Privacy, System Security, Misuse or Bugs**" might encompass a wide range of incidents. Despite this, it was the best choice for enhancing performance with our current dataset. We also knew we could revert to the original taxonomy if needed. In the future, with more data and incidents, we could reapply the first taxonomy to a more balanced dataset and attempt a new model.

Returning to the approaches, we tried the same options than with the original taxonomy, with the difference that we already knew how to prepare the data and adapt the pre-trained model and hyperparameters faster to train the model. Since there are already some explanations about the approaches we already tried with the original taxonomy, now it is time to focus on the results.

With a basic approach of multi-class classification, all the labels in just one column, we could observe that the results were already better than with the original taxonomy, **improving fast up to 0.86** F1-score without much effort. That precision was achieved only by one of our best models with the original taxonomy, but with a lot of overfitting, leading to a really bad performance of the model when introducing new incidents to check its accuracy. We could already say that the new labeling helped the models to have less overfitting.

In our efforts to enhance the model, we achieved a peak **accuracy of 91%** with a similar F1-Score, meeting our objective. However, we were eager to experiment with new approaches using the new taxonomy. Despite the model's impressive metrics, we were concerned about potential overfitting. The training loss was very low, but the validation loss was on the verge of increasing, which is often an indication of overfitting.

We replicated the procedure used with the original labels for the new three labels, aiming to test a multi-labeling approach. We didn't anticipate this method to outperform the straightforward multi-class classification. This is because, with the new labels, it's less likely for incidents to fit into multiple categories. Therefore, we expected fewer instances where the incident descriptions would create uncertainty about their categorization.

We correctly predicted that the performance of multi-labeling with the new taxonomy wouldn't surpass that of multi-class classification. However, it's worth mentioning one particular approach we tried with multi-labeling.

From the beginning of the fine-tuning task, class imbalance was a persistent issue, as evidenced by the numerous mentions throughout this document. There are methods to address this beyond changing labels, such as Under-Sampling and Oversampling. We were not convinced by Under-Sampling, which involves reducing the data in all classes to match the size of the smallest class. While feasible, the idea of using less data in each epoch and repeatedly using the same data from the minority class for balanced training was not ideal for us. This approach could lead to data repetition and underutilization of some labels. Given our existing

issues with overfitting and a limited dataset, we decided against implementing Under-Sampling.

Nonetheless, we experimented with oversampling in our new taxonomy. The label "**Humans Physical Safety**" had fewer instances compared to the other two labels, making it a good candidate for oversampling to achieve balance. It's important to note that oversampling should be done prior to any modifications for fine-tuning. This process is a preliminary step, preceding any transformation of the labels.

For oversampling, we used the Synthetic Minority Over-Sampling Technique (SMOTE) from the "**imblearn**" library. SMOTE is user-friendly; you simply fit your data by separating the target variable from the rest, and the function handles the rest. However, we encountered a challenge: SMOTE doesn't work with text data. It requires a numeric representation of the data to create new samples for balancing.

To address this, we transformed our incident descriptions into a TF-IDF (Term Frequency-Inverse Document Frequency) format using a TFIDF vectorizer. This transformation, however, introduced another issue. The TFIDF vectorizer alters the original text by removing stopwords and merging some expressions. Consequently, when we converted the numerical data back into words or sentences for the model, the descriptions were not exactly like the originals, with some words missing. Despite this, we expected that the essential content of each sentence would still be discernible and retained in the incidents when reverted to text format.

Despite the challenges, we believe it's important to highlight that the final performance using oversampling achieved an accuracy and F1-Score of around 80%-82%. While these figures aren't exceptionally high, there was a notable difference compared to other approaches: there were no signs of overfitting. The training and validation loss decreased proportionally, which is a normal pattern. This suggests that with a more balanced distribution of instances, our results could be less prone to overfitting and potentially more robust when applied to new data.

Inference

Inference is based on testing trained models. The data is pre-processed to match the format used in training, ensuring consistency. This step is crucial to evaluate how the model performs beyond the training and test phases, offering insights into its real-world applicability and effectiveness.

Throughout the training approaches, numerous models were saved. These served not only for later use but also as checkpoints for re-training and as guidelines. For the inference phase, only the models deemed best were utilized. Thanks to inference, some of the models saved were discarded, as overfitting was clearly visible when we introduced self-made incidents or new data to check if the model classified decently.

Saving the models is a straightforward process, as illustrated in **Figure 15**. After the training process, the model and the used tokenizer are saved within the trainer module. This allows for easy reopening and utilization of these trained models, making them ready for immediate application.

```
trainer.save_model("/content/drive/My Drive/Taxonomy_Classification_AI/MULTI-LABEL_fine_tune_0.82_facebook_bart-base_NEW_LABELS_OVERSAMPLED")

tokenizer.save_pretrained("/content/drive/My Drive/Taxonomy_Classification_AI/MULTI-LABEL_fine_tune_0.82_facebook_bart-base_NEW_LABELS_OVERSAMPLED")
```

Figure 15: Saving models

Having saved the model and the tokenizer, all is needed is to open them, and we created our own function for inference:

```
def inference(texts, model=model, tokenizer=tokenizer):
    if type(texts)==str:
        texts=[texts]
    for text in texts:
        encoding = tokenizer(text, return_tensors="pt")
        outputs = model(**encoding)
        logits = outputs.logits #contains scores for each label
        logits.shape #(batch size, num_labels=4)
        # apply sigmoid + threshold
        sigmoid = torch.nn.Sigmoid()
        probs = sigmoid(logits.squeeze().cpu())
        predictions = np.zeros(probs.shape)
        #predictions[np.where(probs >= 0.1)] = 1
        predictions[np.where(probs >=max(probs))]= 1
        print(probs)
        print(predictions)
        # turn predicted id's into actual label names
        # Access the model's config to get the label mapping
        id2label = model.config.id2label
        #print(id2label)
        predicted_labels = [id2label[idx] for idx, label in
enumerate(predictions) if label == 1.0]
    return predicted_labels
```

As you can see, we can open any trained model with its tokenizer, and pass a list of text and it will be returned with their predicted label.

The critical aspect of inference is generating predictions, which are essentially a list showing the probability of each label. In this function, the approach depends on the type of model used:

1. **Multi-Class Model:** If it's a multi-class model, designed to return only one label, the function returns the label with the maximum probability.
2. **Multi-Label Model:** If it's a multi-label model, the approach is different. These models are trained with respect to a threshold. Therefore, the function returns labels based on the threshold set during the training process, which is typically around 0.6.

Here's an example to illustrate this (*Figure 16*):

```
[21] text = "Biased racist opinion of an AI tells fake news about feminism"
      predicted = inference(text)
      print(predicted)

tensor([0.1011, 0.9249, 0.4545], grad_fn=<SigmoidBackward0>)
[0. 1. 0.]
['SOCIAL HARMS']
```

Figure 16: Generating predictions

After completing this process, the decision was made to primarily retain two models. The first one achieves the highest accuracy we have been able to get. The second one, however, even though it is not as great as the first one, doesn't have any signal of overfitting; that is, even if it makes mistakes, we wanted to check how it behaves compared to the model with the best performance metrics. These are the models:

1. “fine_tune_0.91_facebook_bart-base_NEW_LABELS”.
2. “MULTI-LABEL_fine_tune_0.82_facebook_bart-base_NEW_LABELS_OVERSAMPLED”.

If the decision is to choose only one model for use, the preference would be for the first one, which is based on the new labels and has achieved an **accuracy of 0.91**. This model demonstrates impressive performance during inference, accurately classifying both random instances from the dataset and hypothetical incidents created for testing purposes. The few errors it does make typically occur in incidents where categorization is inherently challenging. Overall, the results are highly satisfactory, showcasing strong performance that aligns with the set objectives. Thus, this model is deemed ready for practical application.

Once having obtained the models and their performance, and having checked how they work on inference, it is time to go ahead with the last step, which is to create an interactive visualization plot to display the results and interact with future incidents.

One option recommended by the enterprise was the **Gradio** [15] interface, which is an open-source Python library (offered also by **Hugging Face**) which will predict the label based on the input text. With this, users can interact with the model by entering text into a textbox while the model's predictions are being displayed.

Visualization

Data visualization plays a key role in the field of data science, serving as a powerful tool for transforming raw data into actionable insights. In a discipline driven by huge datasets and complex information growing day by day, visualization acts as the bridge that connects intricate statistical analyses with human understanding.

By representing data in graphical and interactive formats, visualizations offer a compelling means to communicate patterns, trends, and relationships. This not only facilitates the exploration and interpretation of data, but also allows for the effective communication of findings to diverse audiences, regardless of their technical expertise.

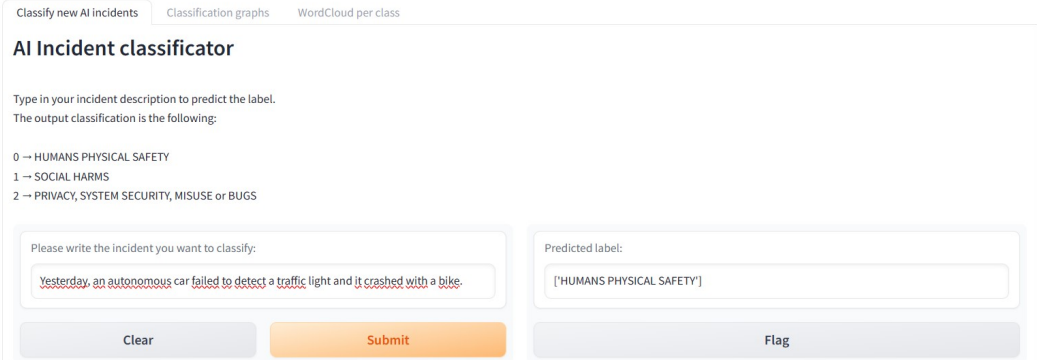
In the realm of AI incident classification, good visualization helps in gaining insights into data patterns and model predictions. In this section, we explore a dynamic and interactive visualization interface built using Gradio [15], a Python library designed for creating user-friendly interfaces around machine learning models.

In previous phases, we made use of models capable of classifying the incidents using LLMs, and as a final part of this project, we aim to present a dashboard which facilitates the creation of a seamless and interactive visualization experience. As a first step, we begin loading the model to the **Gradio interface**. Our dashboard is formed by **three** tabs, each one of them corresponding to a different application

AI Incident Classification

The first tab of our interactive dashboard is dedicated to AI incident classification. A user can input a textual description of an incident, and the interface employs the fine-tuned language model to predict the incident's label. The output classification falls into one of three categories: “**Humans Physical Safety**”, “**Social Harms**”, or “**Privacy, System Security, Misuse or Bugs**” giving as an output the label with the highest score.

To implement it, we used the **predict_label(incident)** function we created when dealing with the model creation and calling the function when launching the Gradio interface by `iface1.launch()`. Below, **Figure 17** shows this feature:



The screenshot displays the 'AI Incident classifier' tab within a Gradio interface. At the top, there are three tabs: 'Classify new AI incidents' (selected), 'Classification graphs', and 'WordCloud per class'. The main area is titled 'AI Incident classifier' and contains instructions: 'Type in your incident description to predict the label. The output classification is the following:'. Below this, a legend lists three categories: '0 -> HUMANS PHYSICAL SAFETY', '1 -> SOCIAL HARMS', and '2 -> PRIVACY, SYSTEM SECURITY, MISUSE OR BUGS'. A text input field is labeled 'Please write the incident you want to classify:' and contains the text: 'Yesterday, an autonomous car failed to detect a traffic light and it crashed with a bike.' Below the input field are 'Clear' and 'Submit' buttons. To the right, a 'Predicted label:' field shows the output: '["HUMANS PHYSICAL SAFETY"]'. A 'Flag' button is located at the bottom right of the prediction area.

Figure 17: AI Incident classifier

Classification graphs

Comprehending the risk distribution within our incident dataset is vital for discerning overall trends. The second tab provides an interactive interface where users can choose between plots such as bar plots or pie charts to see the quantity of incidents that fall into the 3 existing labels.

For the generation of plots we made use of the interactive library **Plotly Express** [16] from **Dash**. This dynamic approach allows for a visual exploration of the data we worked with during this project. Let **Figure 18** show an example:

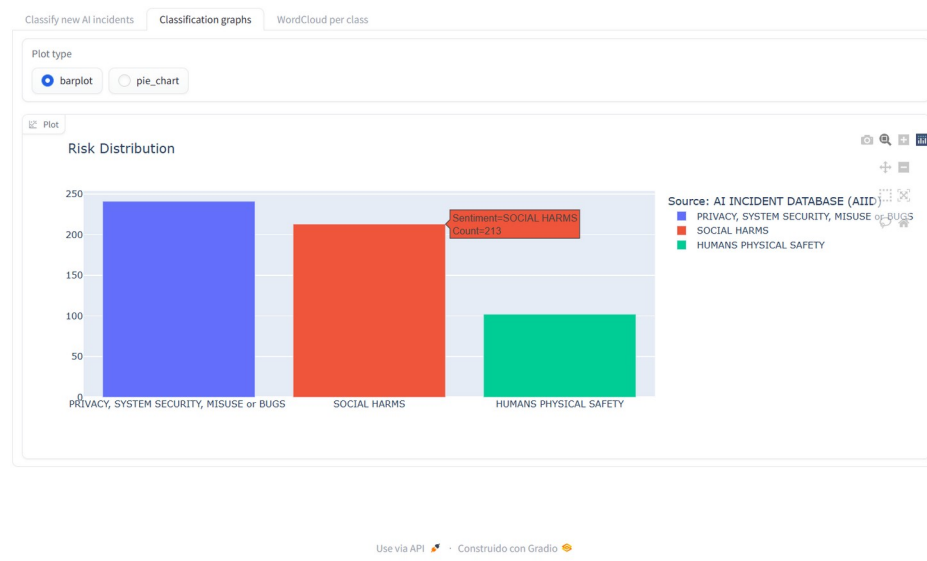


FIGURE 18. RISK DISTRIBUTION

It could be noticed also that a pie chart can also be generated (**Figure 19**). As we can see, **SOCIAL HARMS** incidents represent a **38.3%** of the total entries in the database, **PRIVACY, SYSTEM SECURITY, MISUSE or BUGS** a **43.3%**, and **HUMANS PHYSICAL SAFETY** a **18.3%**, being the least frequent label in our database. From this we can conclude that incidents concerning privacy and system flaws are the most reported ones.

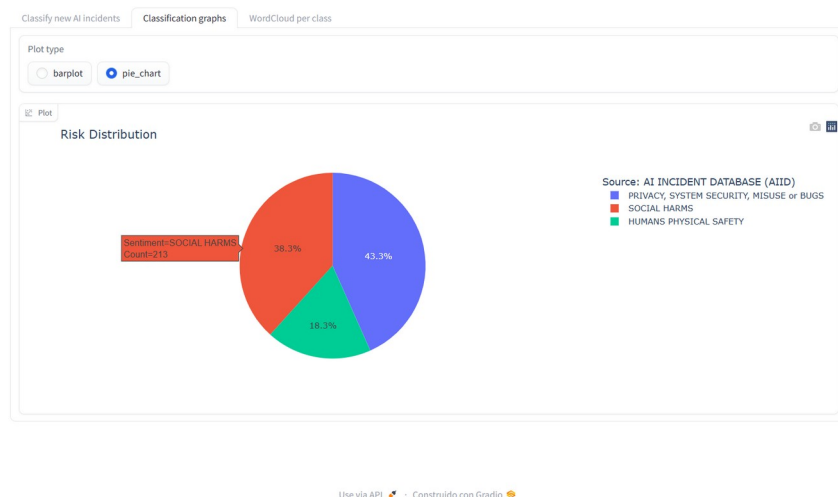


Figure 19: Pie chart

WordCloud per class

To delve deeper into the specifics of each risk, the third tab enables users to select a particular taxonomy class through a dropdown menu and generate a word cloud for each label. This graphical representation showcases the most frequent words associated with the chosen label, offering a visually intuitive way to comprehend the language and themes prevalent in each label.

Let's take a look at **Figure 20**. Regarding the label “**HUMANS PHYSICAL SAFETY**”, some of the most appearing words tend to be **Tesla, autonomous, car, driver** and **system**, reflecting in this way incidents that threaten people's physical safety.

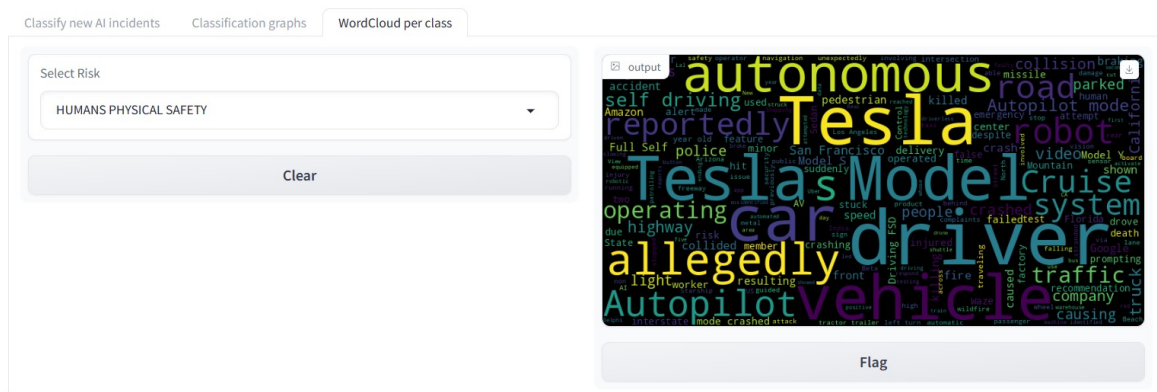


Figure 20: WordCloud (ex.1)

For the label “**SOCIAL HARMS**”, *Figure 21* shows that **Facebook, women, racial, video, algorithm** represent mostly this kind of incidents related to a social environment harm.

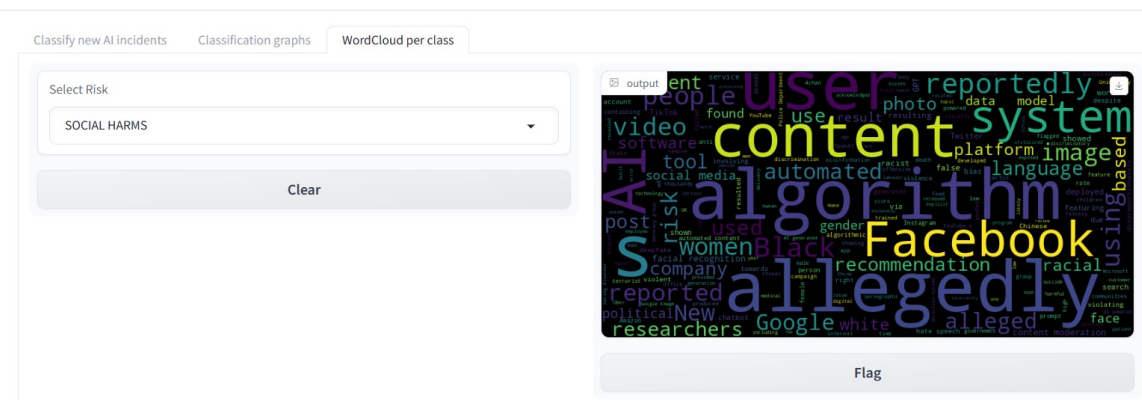


Figure 21: WordCloud (ex.2)

Finally, for the last label (**PRIVACY, SYSTEM SECURITY, MISUSE or BUGS**), some of the repeated words (*Figure 22*) are **facial, recognition, AI, privacy, image,**

reportedly, which are related to those incidents that include privacy violation, bugs, system flaws and illegal data access.

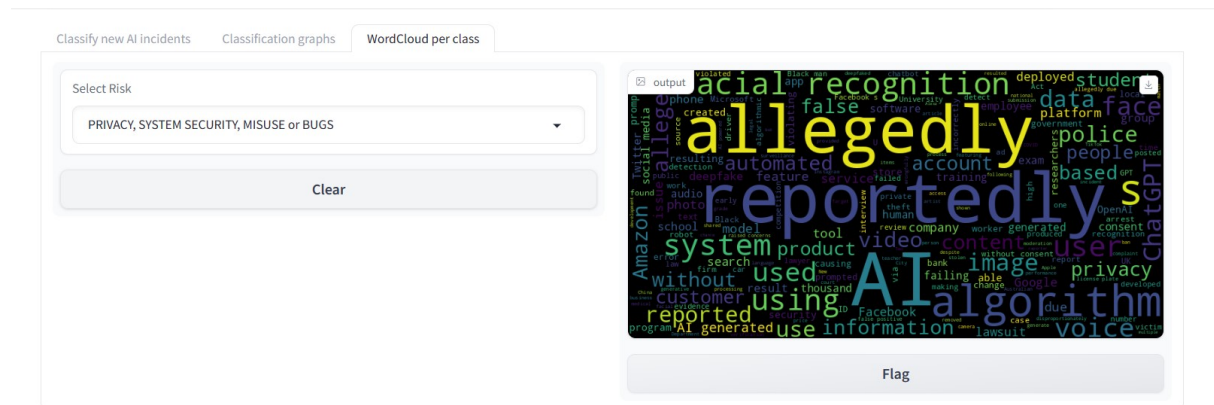


Figure 22: WordCloud (ex.3)

Alternatives

When exploring the alternatives of deploying our project's interface, **Streamlit** and **Gradio** stand out as compelling options, each with its set of advantages and drawbacks.

Streamlit offers an impressive level of customization, granting developers extensive control over interface design, layout, and interactions. This flexibility is particularly advantageous when dealing with complex layouts or requiring advanced components and interactions. However, mastering Streamlit might demand a steeper learning curve. Achieving more intricate designs or dynamic updates might necessitate a deeper understanding of its internal mechanisms.

Conversely, Gradio prioritizes simplicity and user-friendliness. Its API is tailored for swift deployment and rapid prototyping of machine learning models, making it especially appealing for beginners or those seeking quick deployment without extensive coding. While Gradio excels in simplicity and speed, it might have limitations in terms of customization when compared to Streamlit. Its pre-built UI components simplify development but might restrict flexibility for more complex user interactions or layouts.

Our decision to utilize Gradio was influenced by several factors. Its ease of implementation with Hugging Face models specifically, along with its user-friendly deployment options on the Hugging Face spaces section, were pivotal reasons behind choosing Gradio for our project's interface.

Conclusions

In navigating the landscape of AI incident analysis and classification, our journey led us through meticulous iterations, from taxonomy refinement to model training and interactive visualization. Recognizing the complexities of diverse incident categories and the challenges

posed by class imbalances, we streamlined our classification system into three encompassing labels:

1. Human Physical Safety.
2. Social Harms.
3. Privacy, System Security, Misuse, or Bugs.

The adaptation of the taxonomy proved instrumental in enhancing model performance. Through systematic approaches and adaptations to model architectures, our pursuit aimed at mitigating overfitting while maximizing accuracy. Experimentation with multi-class and multi-label classification strategies unveiled insights into handling imbalanced datasets, where oversampling using SMOTE, albeit challenging with text data, showed promise in reducing overfitting tendencies and offering robustness to new data.

The inference pipeline, carefully designed to handle model predictions, facilitated model selection. Prioritizing robustness over slightly higher accuracy, we identified models that, while potentially making marginally more errors, showcased resilience against overfitting, ensuring reliability in real-world scenarios.

Our commitment to effective communication led us to leverage Gradio for interface development, offering users an intuitive and interactive platform to input incident descriptions and receive model predictions. Plotly Express further enriched the user experience by enabling dynamic visualization of risk distributions and word clouds, unraveling insights within the incident dataset.

This holistic journey from taxonomy refinement to model deployment underscored the significance of adaptability, robustness, and user-friendliness in the realm of AI incident analysis. While achieving a 91% accuracy milestone, our emphasis on model resilience and interpretability remains paramount, ensuring reliable insights in real-world contexts. With this project, we've not only refined incident classification but also crafted an interactive interface, empowering users to engage with AI incident data more effectively.

In the constantly evolving landscape of AI and its societal impacts, our work stands as a testament to the nuanced approach required in understanding, classifying, and communicating incidents, paving the way for more informed and responsible AI utilization.

In addition to the theoretical and practical insights shared in this report, we have made the project's codebase accessible. The complete code, including data preprocessing scripts, model training routines, and visualization tools, is available in our [Google Drive folder](#).

Next steps

As we chart the course for the future of the project, we have outlined several potential next steps that hold the promise of enhancing its capabilities and user experience. While these steps are not set in stone, they represent exciting possibilities that can elevate our project to new heights. These are some of the ideas:

1. **Automated Model Retraining:** Implementation of a system that periodically fetches the latest dataset available, setting up a pipeline for automated data preprocessing, model retraining, and evaluation. We also consider incorporating

techniques like transfer learning or online learning to adapt to evolving patterns in the data.

2. **Incident Submission to Database:** Development of a secure and user-friendly interface for incident submission, implementing validation checks to ensure the submitted incidents adhere to the defined taxonomy. We considered implementing a moderation system to review and validate user-submitted incidents before adding them to the database.
3. **Dashboard Enhancement:** Another option under consideration is the evaluation of Streamlit's features and customization capabilities. Depending on our UI/UX requirements, we may explore migrating our existing Gradio-based dashboard to Streamlit. Additionally, we are contemplating the addition of new visualizations and insights for users.
4. **Additional Data Exploration:** We considered incorporating more detailed statistics or visualizations that provide insights into model performance, such as precision-recall curves, confusion matrices, or feature importance. Another idea that came to our mind was to integrate mechanisms to track model drift over time to ensure continuous monitoring of performance.
5. **User Feedback and Iterative Improvement:** Collection of user feedback on the current system and dashboard, implementing features that allow users to provide feedback on model predictions or dashboard usability. This way, we can iteratively improve the system based on user feedback and emerging needs.
6. **Scalability and Performance Optimization:** Assessment of the scalability of the system, especially if the dataset grows significantly over time. Another idea is to optimize the performance of the model and inference pipeline to handle increased user interactions.

References

- [1] "Ethics Guidelines for Trustworthy AI," Shaping Europe's Digital Future, 2019. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai>. [Accessed Jan. 4, 2024].
- [2] "Proposal for a Regulation Laying Down Harmonised Rules on Artificial Intelligence," Shaping Europe's Digital Future, 2021. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/library/proposal-regulation-laying-down-harmonised-rules-artificial-intelligence>. [Accessed Jan. 4, 2024].
- [3] "EU AI Act: First Regulation on Artificial Intelligence," European Parliament News, 2023. [Online]. Available: <https://www.europarl.europa.eu/news/en/headlines/society/20230601STO93804/eu-ai-act-first-regulation-on-artificial-intelligence>. [Accessed Jan. 4, 2024].

- [4] AI Incident Database," AI Incident Database RSS. [Online]. Available: <https://incidentdatabase.ai/>. [Accessed Nov. 9, 2023].
- [5] "Draft - Taxonomy of AI Risk," National Institute of Standards and Technology, 2021. [Online]. Available: https://www.nist.gov/system/files/documents/2021/10/15/taxonomy_AI_risks.pdf. [Accessed Nov. 10, 2023].
- [6] M. Hoffman, M. Narayanan, Y.-J. Liao, and H. Frase, "CSET's Harm Taxonomy for the AI Incident Database," GitHub. [Online]. Available: <https://github.com/georgetown-cset/CSET-AIID-harm-taxonomy/blob/main/README.md>. [Accessed Nov. 9, 2023].
- [7] R. Shelby et al., "Sociotechnical Harms of Algorithmic Systems: Scoping a Taxonomy for Harm Reduction," arXiv, 2022. [Online]. Available: <https://arxiv.org/pdf/2210.05791.pdf>. [Accessed Nov. 10, 2023].
- [8] "NLTK," Natural Language Toolkit. [Online]. Available: <https://www.nltk.org/>. [Accessed Nov. 9, 2023].
- [9] "Gensim: Topic Modelling for Humans," Radim Řehůřek: Machine Learning Consulting. [Online]. Available: <https://radimrehurek.com/gensim/index.html>. [Accessed Nov. 9, 2023].
- [10] "What is Zero-Shot Classification? - Hugging Face." [Online]. Available: <https://huggingface.co/tasks/zero-shot-classification>. [Accessed Nov. 9, 2023].
- [11] Facebook/Bart-Large-mnli, <https://huggingface.co/facebook/bart-large-mnli> (accessed Nov. 9, 2023).
- [12] Facebook/Bart-Base, <https://huggingface.co/facebook/bart-base> (accessed Nov. 20, 2023).
- [13] "Transformers Tutorials for Multi Label Text Classification," Niels Rogge on GitHub. [Online]. Available: [https://github.com/NielsRogge/Transformers-Tutorials/blob/master/BERT/Fine_tuning_BERT_\(and_friends\)_for_multi_label_text_classification.ipynb](https://github.com/NielsRogge/Transformers-Tutorials/blob/master/BERT/Fine_tuning_BERT_(and_friends)_for_multi_label_text_classification.ipynb). [Accessed Dec. 5, 2023].
- [14] "Label Studio." [Online]. Available: <https://labelstud.io/>. [Accessed Dec. 5, 2023].
- [15] "Gradio," Gradio Team. [Online]. Available: <https://www.gradio.app/>. [Accessed Nov. 9, 2023].
- [16] "Plotly Express," Plotly, 2023. [Online]. Available: <https://plotly.com/python/plotly-express/>. [Accessed Dec. 5, 2023].